

Chapter Five

Modular Programming

Modularity

- Modular programming
 - Breaking down the design of a program into individual components (modules) that can be programmed and tested independently
 - A program divided into several smaller parts which can interact
- Modules
 - Can be written and tested separately
 - Testing is easier (smaller)
 - Doesn't need to be retested
 - Reduces length of program
 - Hides details (abstraction)

Functions in C++

- Modules in C++ are called functions
- Function - a subprogram that can act on data and return a value
- Every C++ program has at least one function, `main()`, where program execution begins
- A C++ program might contain more than one function.
- Functions may interact using function call
- Types of C++ Functions :
 - user-defined : defined by a programmer/user.
 - built-in : already defined in C++ software.
 - E.g `pow()`, `sqrt()`, `abs(x)`, `log(x)`, `sin(x)`, `floor(x)`, `strcpy()`, `strlen(s)`, `cin`, etc.

Declaration of Functions

- Functions must be declared before use
- The declaration tells the compiler
 - return type,
 - The name,
 - parameters of the function
 - E.g. `int add(int x,int y), void display( string name)`
- Three ways:
 - Write your prototype into a separate file, and then use the `#include` directive to include it in your program.
 - Write the prototype into the file in which your function is used.
 - Define the function before it is called by any other function.

Function Prototypes

- The declaration of a function is called its prototype
- Is a statement - it ends with a semicolon
- It consists of the function's
 - return type,
 - name,
 - parameter list
- **Syntax**
 - *return_type function_name (type [parameterName1], type [ParameterName2] ...);*
 - *E.g. long Area(int, int);*

Or

long Area(int length, int width);

Example of function definition

- **//Function with no arguments, no return value**

```
• void greet()
{
    cout << "Hello!";
}
```

- **//Function with arguments, no return value**

```
• void printSum(int a, int b)
{
    cout << a + b;
}
```

- **//Function with arguments and return value**

```
• int square(int x)
{
    return x * x;
}
```

- **//Function with no arguments but return value**

```
• int getNumber()
{
    return 10;
}
```

Function Prototypes

- If the function doesn't have a return type void will be used
 - void is a reserved word
- The function prototype usually placed at the beginning of the program
- Many of the built-in functions have their function prototypes already written in the *header* files you include in your program by using `#include`

Defining a Function

- The definition tells the compiler how the function works.
- Consists of :
 - the function header :
 - like the function prototype except that the parameters must be named
 - there is no terminating semicolon
 - its body
 - the task of the function

- **Syntax**

```
    return_type function_name (parameter
declarations)
{
    declarations;
    statements;
}
```


Defining a Function

- *E.g. function to compute area of a rectangle.*

```
long Area(long l, long w)
```

```
{
```

```
    return l * w;
```

```
}
```

- The return statement without any value is typically used to exit the function early
- **C++** does not allow nested functions
 - The definition of one function cannot be included in the body of another function
- A function definition must agree in return type and parameter list with its prototype

Defining a Function

- If the function returns a value, it's definition should end with a *return* statement
- The body of the function is always enclosed in braces, even when it consists of only one statement
- **return** keyword
 - Returns data, and control goes to function's caller
 - If no data to return, use **return**;
 - Function ends when reaches right brace
 - Control goes to caller

- Early Exit Using return: to **exit a function early** when a condition is met.
- ```
void checkAge(int age)
{
 if (age < 18)
 return; // exit early
 cout << "Access granted";
}
```
- returning a **negative integer** mean: depends on the context in which the function is used.
- ```
int checkValue(int x) {
    if (x < 0)
        return -1; // negative integer
    return 1;
}
```
- Negative Return Values as **Error Codes** (Very Common)
- ```
int divide(int a, int b) {
 if (b == 0)
 return -1; // error: division by zero
 return a / b;
}
```

# What must agree between function **prototype**, **definition**, and **call**?

- 1. **Function name**
- 2. **Number of parameters**
- 3. **Type of parameters**
- 4. **Order of parameters**
- 5. **Return type**

```
• int sum(int, float); // prototype
int sum(int a, float b) { // definition
 return a + b;
}
sum(3, 4.5); // call
```

| Must agree           | Prototype | Definition | Call           |
|----------------------|-----------|------------|----------------|
| Function name        | ✓         | ✓          | ✓              |
| Number of parameters | ✓         | ✓          | ✓              |
| Parameter types      | ✓         | ✓          | ✓              |
| Order of parameters  | ✓         | ✓          | ✓              |
| Return type          | ✓         | ✓          | Used correctly |
| Parameter names      | ✗         | ✗          | ✗              |

```
// Creating and using a programmer-defined function.
#include <iostream.h>

int square(int); // function prototype

int main()
{
 // loop 10 times and calculate and output
 // square of x each time
 for (int x = 1; x <= 10; x++)
 cout << square(x) << " "; // function call

 cout << endl;

 return 0; // indicates successful termination

} // end main
```

Function prototype: specifies data types of arguments and return values.  
**square** expects an **int**, and returns an **int**.

Parentheses ( ) cause function to be called.  
When done, it returns the result.

```
// square function definition returns square of an integer
int square(int y) // y is a copy of argument to function
{
 return y * y; // returns square of y as an int
} // end function square
```

Definition of **square**. **y** is a copy of the argument passed.  
Returns **y \* y**, or **y** squared.

1   4   9   16   25   36   49   64   81   100

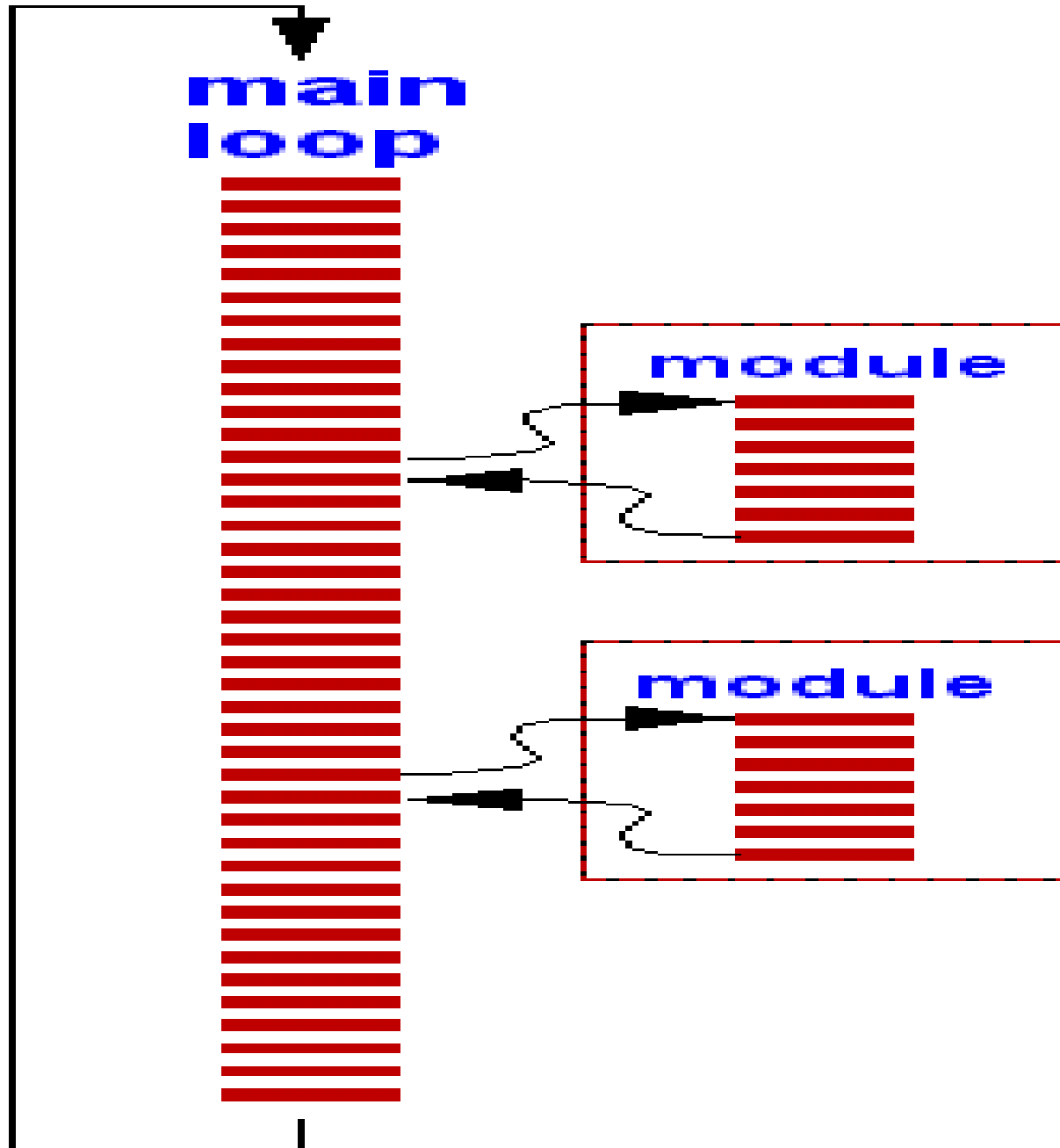
# Program Using a Function

```
#include <iostream.h>
double Celsius_to_Fahr(double); //Function Prototype
int main()
{
 double temp,result;
 cout<<"enter the temperature"<<endl;
 cin>>temp;
 result= Celsius_to_Fahr(temp);
 cout<<"the corresponding Fahrenheit
is"<<result<<endl;

double Celsius_to_Fahr(double Celsius)
{
 double temp; // Declare variables
 temp = (9.0/5.0)*Celsius + 32; // Convert
 return temp;
}
```

# Execution of Functions

- Each function has its own name
- When that name is encountered, called function call, the execution of the program branches to the body of that function – called function
- When the function returns, execution resumes on the next line of the calling function
- Functions can also call other functions and can even call themselves





# Exercise

- Write modular program for arithmetic operations
  - add()
  - subtract()
  - multiply()
  - divide()
- Write a modular program to compute area of a rectangle, square and circle.

Write a program that uses **three separate functions**:

- inputNumbers()
- calculateSum(int a, int b)
- displayResult(int sum)

Write a program using the following modules:

- readArray(int arr[], int n)
- findMax(int arr[], int n)
- findMin(int arr[], int n)
- displayResults(int max, int min)

# ***Scope of identifier***

- Refers to where in the program an identifier is accessible
- Determines where it can be accessed
- Two kinds
  - Local identifier - identifiers declared and used within a function (or block).
  - Global identifier – identifiers declared outside of every function definition.

# Local/block scope

- You can declare variables within the body of the function
  - local variables
  - When the function returns, the local variables are no longer available
- Variables declared within a block are scoped to that block – Local to that block
  - they can be accessed only within that block and "go out of existence" when that block ends
  - E.g.

```
for(int i = 0;i<5; i++)
{
 cout<<i;
}
```

```
i=+10; // compilation error i is inaccessible
```

# Local variable

- ```
void func() {  
    int x = 10; // local to func  
    if (x > 5)  
{  
        int y = 20; // local to if block  
        cout << y;  
    }  
    cout << y; ✗ ERROR: y is out of scope  
}
```

Global Scope

- Variables defined outside of any function have global scope.
- Available from any function in the program, including *main()*
- A local variable with the same name as a global variable **hides the global variable - when used within the function.**

```
#include <iostream.h>
void myFunction(); // prototype
int x = 5, y = 7; // global
    variables
int main()
{
    cout << "x from main: " << x << "\n";
    cout << "y from main: " << y << "\n";
    myFunction();
    cout << "Back from
        myFunction!\n\n";
    cout << "x from main: " << x << "\n";
    cout << "y from main: " << y << "\n";
    return 0;
}
```

```
void myFunction()
{
    int y = 10;
    cout<<"x from myFunction:"<<x<<"\n";
    cout<<"y from myFunction:"<<y<<"\n";
}
```

Output:

```
x from main: 5
y from main: 7
  x from myFunction: 5
  y from myFunction: 10
Back from myFunction!
x from main: 5
y from main: 7
```

Global/Local Example

```
int count(0); //is the same as int count = 0;
...
int main()
{
    int x, y, z;
    count-=y+z;
    ...
}
int calc(int a, int b)
{
    int x;
    count += x;
    ...
}
```

Unary Scope Resolution Operator ::

- Using ::, one can access a global variable even if it is over-shadowed by a local variable of the same name.
- E.g.

```
#include <iostream.h>
float num=10.8;
int main(){
float num= 9.66;
cout<<"the value of num is:"<<::num<<endl;
return 0;
}
```

Output:
the value of num is: 10.8

- `#include <iostream>`
`using namespace std;`
`int x = 10; // global variable`

`int main()`
`{`
 `int x = 20; // local variable`
 `cout << x << endl; // 20 (local)`
 `cout << ::x << endl; // 10 (global)`
`}`

inline Functions

- An **inline function** is a function where the compiler **replaces the function call with the actual function code** at compile time, instead of performing a normal function call.
- Function calls
 - Cause execution-time overhead
- Qualifier **inline** before function return type "advises" a function to be inlined
 - Puts copy of function's code in place of function call
 - Speeds up performance but increases file size
 - Compiler can ignore the **inline** qualifier
 - Ignores all but the smallest functions
 - E.g.

```
inline double cube( const double s )  
{  
    return s * s * s;  
}
```

inline functions

- Advantage: function call overhead is eliminated, thus faster and less memory consuming
- Disadvantage: the code is expanded during compilation so that executable file is large.

Storage class

- Storage class determines the period during which an identifier exists in memory.
- Four storage classes
 - auto
 - Local variables in functions or blocks. Auto storage class variables are created only when the block is active, and disappear when the block or function exits.
 - register
 - Variable existence like auto. The register variable is a suggestion to the compiler to put the variable in a CPU register.
 - static
 - Local static variable exists during the whole program executing, i.e., the variable retains its value between function calls.
 - However, the reference to the variable is local in the function or block
 - extern
 - Global variables and function names have the storage class extern. Extern storage class variable exists during the whole program execution.

Storage class and variables

- Automatic variable - memory is allocated at block entry and deallocated at block exit.
- Static variable - memory remains allocated as long as the program executes
 - Retain their values across function calls.
- **Variables** declared outside of any block are static **variables**
- By default, **variables** declared within a block are automatic **variables**
- Declare a static variable within a block by using the reserved word **static**.

Static and Automatic **Variables** (continued)

- The syntax for declaring a static variable is:
static dataType identifier;
 - E.g.
static int x;
 - declares x to be a static variable of the type int
- Static **variables** declared within a block are local to the block
- Their **scope** is the same as any other local identifier of that block but life span is throughout a program execution.
- **Behavior of a static variable:**
 - **Scope:** Local to the function
 - **Lifetime:** Entire program
 - **Initialization:** Happens **only once**
 - **Value is preserved** between function calls

- ```
void counter() {
 static int x = 0;
 x++;
 cout << x << endl;
}
```

## extern

- Used to access a **global variable defined in another file**  

```
// file1.cpp int x = 10; // file2.cpp
extern int x;
```

| Storage Class | Scope        | Lifetime       |
|---------------|--------------|----------------|
| auto          | Block        | Block          |
| register      | Block        | Block          |
| static        | Block / File | Entire program |
| extern        | Global       | Entire program |

# Storage Class and Scope

- Local Objects
  - Defined within a function
  - Can be accessed only within the function
  - Automatic storage class (default)
  - Static storage class (retained entire execution)
- Global Objects
  - Defined outside all functions
  - Can be accessed by any function
  - External storage class



- ```
#include <iostream.h>
void echo1();
    int main(){
    echo1();
}
    void echo1()
{
    for(int i=0;i<2;i++)
    {
        int num =0;
        cout<<num<<endl;
        num++;
    }
```

Functions with Default Parameters

- When a function is called
 - The number of **actual** and **formal** parameters must be the same
- **C++** relaxes this condition for functions with default parameters.
- You specify the value of a default parameter when the function name appears for the first time, such as in the prototype or function header.

Syntax of default parameter:

```
return_type function_name(type1 param1 = default value1,  
type2 param2 = default value2, ...)
```

```
{  
    // function body  
}
```

- Example

- ```
void print(int x, int y = 0) {
 cout << "x = " << x << ", y = " << y << endl;
}
```

```
int main() {
 print(5); // y defaults to 0 → x=5, y=0
 print(5, 10); // x=5, y=10
}
```

# Functions with Default Parameters (continued)

- If you do not specify the value of a default parameter
  - The default value is used
- All of the default parameters must be the rightmost parameters of the function.
- In a function call where the function has more than one default parameter and a value to a default parameter is not specified
  - You must omit all of the arguments to its right
- Default values can be constants, global **variables**, or function calls.
- The caller has the option of specifying a value other than the default for any default parameter
- You cannot assign a constant value as a default value to a reference parameter

- `void func(int a = 1, int b = 2, int c = 3);`
- `func(10);`    `// a=10, b=2, c=3`
- `func(10, 20);` `// a=10, b=20, c=3`
- `// func( , 20);` **✗** Not allowed

# Empty Parameter Lists

- functions can take no arguments
- To declare that a function takes no parameters:
  - Write **void** or nothing in parentheses
    - E.g
      - `void print1( void );`
      - `void print2();`

# Parameter Passing

- Call by Value
  - **Value** of the function argument passed to the formal parameter of the function
  - Copy of data passed to function
  - Changes to copy do not change original variable
- Call by Reference
  - **Address** of the function argument passed to the formal parameter of the function.
  - In **call by reference**, the function receives a **reference (alias)** of the actual argument.
  - Changes made inside the function **affect the original variable**.

# Call by Reference

- If a formal parameter is a reference parameter
  - It receives the address of the corresponding actual parameter
  - A reference parameter stores the address of the corresponding actual parameter
- During program execution to manipulate the data
  - The address stored in the reference parameter directs it to the memory space of the corresponding actual parameter
  - A reference parameter receives the address of the actual parameter
- Reference parameters can:
  - Pass one or more values from a function
  - Change the value of the actual parameter



# Call by Reference

- Reference parameters are useful in three situations:
  - Returning more than one value
  - Changing the actual parameter
  - When passing the address would save memory space and time

```

#include <iostream.h>
int squareByValue(int);
void squareByReference(int &);
int main()
{
 int x = 2, z = 4;
 cout << "x = " << x << " before squareByValue\n" << "Value
 returned by squareByValue:" << squareByValue(x)<<endl
 << "x = " << x << " after squareByValue\n" << endl;
 cout << "z =" << z << " before squareByReference" <<
 endl;
 squareByReference(z);

 cout << "z = " << z << " after squareByReference" <<
 endl;
 return 0;
}
int squareByValue(int a)
{
 return a *= a; // caller's argument not modified
}
void squareByReference(int &cRef)
{
 cRef *= cRef; // caller's argument modified
}

```

# Call by Value Example

```
/* Incorrect function to switch two values */

void swap(int a, int b)
{
 int hold;

 hold = a;
 a = b;
 b = hold;

 return;
}
```

# Call by Reference Example

```
/* Correct function to switch two values */

void swap2(int& a, int& b)
{
 int hold;

 hold = a;
 a = b;
 b = hold;

 return;
}
```

## Pass by value

- `#include <iostream>`
- `using namespace std;`

```
void increment(int x)
{
 x = x + 1;
}
```

- `int main()`

```
{
 int num = 5;
 increment(num);
 cout << num; // Output: 5
 (original value is not
 changed)
```

## Pass by Reference

- `#include <iostream>`

```
using namespace std;
void increment(int &x) { // x is
 a reference to num
 x = x + 1;
 }
```

- `int main()`

```
{
 int num = 5;
 increment(num);
 cout << num; // Output: 6
 (original value changed)
}
```

# Recursion and Recursive Functions

- Main calls another function.....normal
  - A function calls another function2....normal
  - A function calls itself ?! Possible?? **YES**
- A recursive function is one that call itself.

# Concept Of recursion

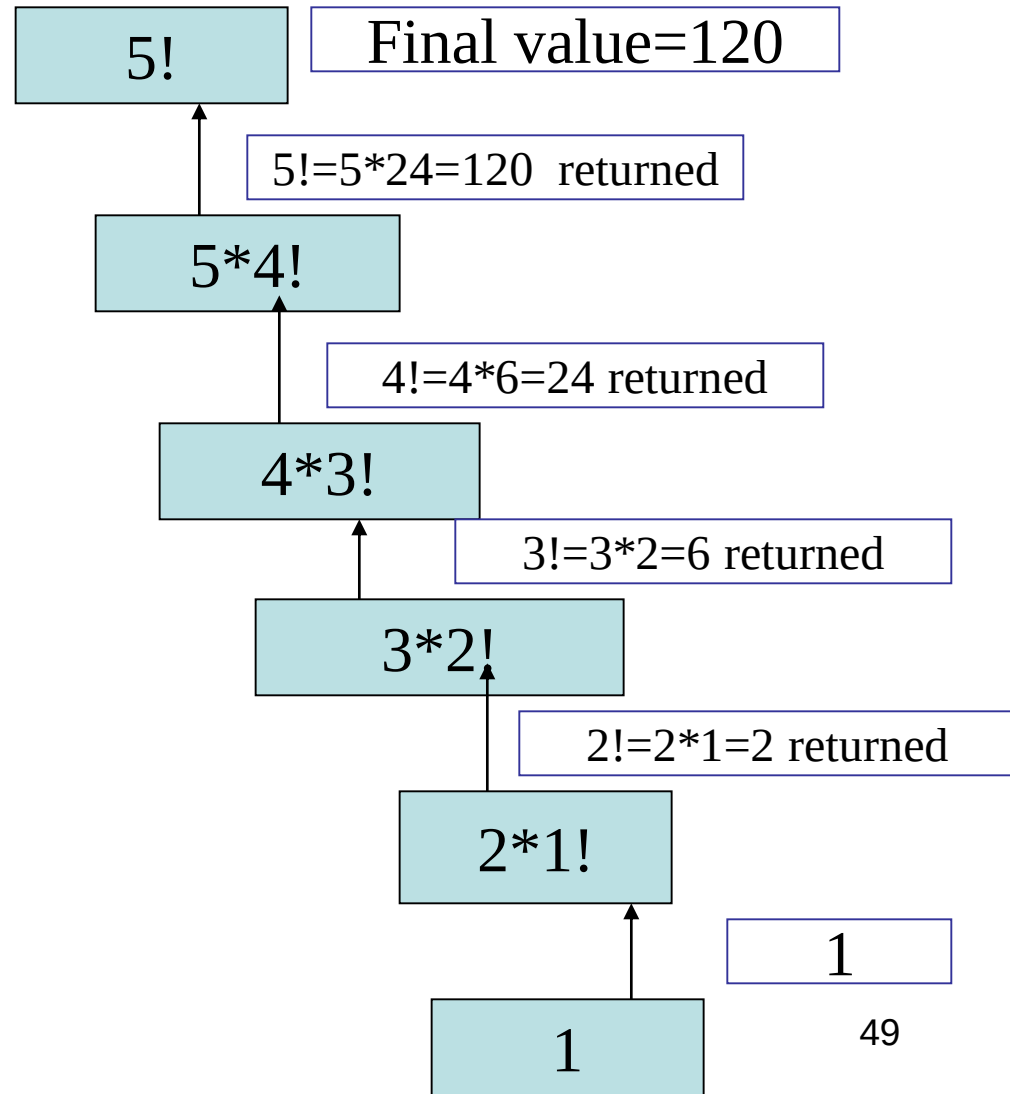
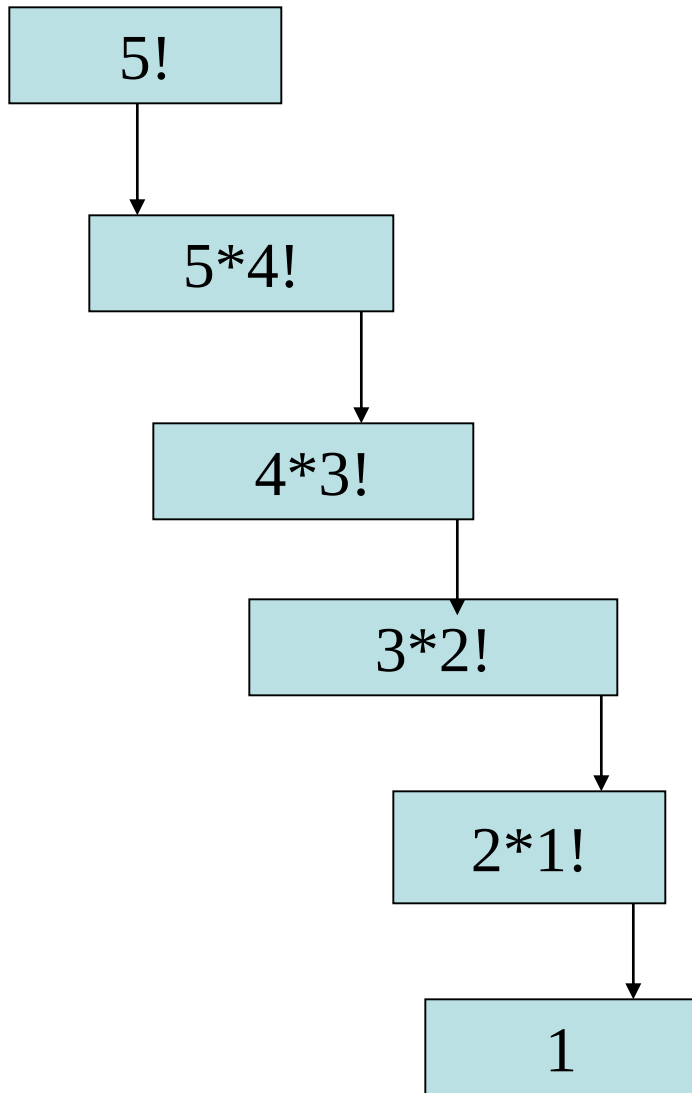
- A function can also call itself, this is known as recursion
- To avoid infinite recursion, one must have a terminating condition in the function
- A recursive function is called to solve a problem
- The function knows to solve only the simplest cases or so-called **base-cases**
- Thus if the function called with a base-case, it simply returns a result. But if it is called with more complex problem, the function divides the problem into two conceptual pieces, one knows how to do, and another doesn't know what to do.
- The second case/piece must resemble the original problem, but be a slightly simpler/smaller version of the original problem

# Concept Of recursion (cont.)

- Thus the function launches (calls) a fresh copy of itself to work on the smaller problem –**this is related as a Recursive-call/recursive step.**
- The function keeps dividing each new sub problem into two conceptual pieces until eventually terminates after converging on the **base-case.**
- The function thus recognize the base-case and returns a result to the previous copy of the way up the line until original call of the function returns the final result to main.
- **Key requirements:**
  - **Base case:** Stops the recursion to prevent infinite calls.
  - **Recursive case:** Function calls itself with modified arguments, moving toward the base case.



# Finding Factorial Recursively



# Finding Factorial iteratively

```
#include<iostream.h>
#include<iomanip.h>
unsigned long factorial(unsigned long) ;//prototype
int main()
{
 int num;
 cout<<"enter a positive integer:";
 cin>>num;
 cout<<"factorial="<<factorial(num) ;
 return 0;
}
unsigned long factorial(unsigned long n)
{
 unsigned long fact = 1;
 For(int i=1; i<=n; i++)
 fact*=i;
}
```

# Finding Factorial Recursively

```
//Recursive factorial Function
#include<iostream.h>
#include<iomanip.h>
unsigned long factorial(unsigned long); //prototype
int main()
{
 int num;
 cout<<"enter a positive integer:";
 cin>>num;
 cout<<"factorial="<<factorial(num);
 return 0;
}
unsigned long factorial(unsigned long n)
{
 if (n <= 1) //the base case
 return 1;
 else
 return n * factorial (n - 1);
}
```

# Function Overloading

- Function overloading
  - Functions with same name and different parameters
  - Should perform similar tasks
    - I.e., function to square **ints** and function to square **floats**

```
int square(int x) {return x * x;}

float square(float x) { return x * x; }
```
- A call-time c++ compiler selects the proper function by examining the number, type and order of the parameters

- Valid Function Overloading :
  - Different number of parameters
  - Different parameter types
  - Different order of parameter types
- But not by:
  - different return types only.
  - Different parameter name
  - Value vs reference only
- Example:
  - `void show(int a);`
  - `void show(int a, float b);`      `// different number of parameters`
  - `void show(float a);`      `// different type`
  - `void show(float a, int b);`      `// different order`

# Function overloading

```
#include<iostream.h>
int add(int a, int b) {
 return a + b;
}
int add(int a, int b, int c) {
 return a + b + c;
}
double add(double a, double b) {
 return a + b;
}
int main() {
 cout << add(2, 3) << endl;
 cout << add(2, 3, 4) << endl;
 cout << add(2.5, 3.5) << endl;
 return 0;
 • }
```

**1. Cannot overload by return type only:**

```
int func(int a); double func(int x);
// Error: same parameter list
```

**2. Works for:**

- Different **number of parameters**
- Different **types of parameters**
- Combination of both

**3. Function overloading helps readability and code reuse.**

# Ambiguous Overloading

- `void fun(int a);`
- `void fun(float a);`
- `fun(5.5);` // **×** ambiguous
- 5.5 is of type **double** (by default in C++)
- There is **NO function** with a double parameter.
- The compiler looks for the **best match** and considers **type conversions**:
  1. double → int
  2. double → float
- **Why is it ambiguous?**
  - Neither function is an **exact match**
  - Neither conversion is **better** than the other
  - The compiler **cannot decide** which function to call
- Therefore, the call is **ambiguous**.

# Classwork

1. Write three overloaded functions named Area() to compute:
  - Area of a rectangle
  - Area of a circle
  - Area of a square
  - Call each version from main() and display the result.
2. Create overloaded functions maxValue() to find the maximum of:
  - two integers
  - two doubles
  - three integers
3. Write functions to compute square of a number by passing a value and reference.
4. For each function call, identify whether it uses **call by value** or **call by reference**:
  - void f1(int x);
  - void f2(int &x);
  - int a = 5;
  - f1(a);
  - f2(a);



- 5. Can function overloading differ only by return type? Why?
- 6. When should call by value be preferred?
- 7. why the following code is ambiguous for the compiler?

```
#include <iostream>
```

```
using namespace std;
```

```
void compute(int a, long b)
```

```
{
```

```
 cout << "compute(int, long) called" << endl;
```

```
}
```

```
void compute(long a, int b)
```

```
{
```

```
 cout << "compute(long, int) called" << endl;
```

```
}
```

```
int main()
```

```
{
```

```
 compute(10, 20);
```

```
return 0;
```

```
}
```

# Add overloading

```
• #include <iostream>

/* Area of a rectangle */
float Area(float length, float width)
{
 return length * width;
}

/* Area of a circle */
float Area(float radius)
{
 return 3.14159f * radius * radius;
}

/* Area of a square */
float Area(int side)
{
 return side * side;
}
```

```
• int main()
{
 float length = 5.0, width = 3.0;
 float radius = 4.0;
 int side = 6;

 cout << "Area of rectangle = " <<
 Area(length, width) << endl;
 cout << "Area of circle = " <<
 Area(radius) << endl;
 cout << "Area of square = " <<
 Area(side) << endl;
 return 0;
}
```

# square

- `#include <iostream>`

`/* Call by Value */`

```
void squareByValue(int x)
{
 x = x * x;
 cout << "Inside squareByValue(): x
= " << x << endl;
}
```

`/* Call by Reference */`

```
void squareByReference(int &x)
{
 x = x * x;
 cout << "Inside
squareByReference(): x = " << x <<
endl;
}
```

```
int main()
```

```
{
```

```
 int num = 5;
```

```
 cout << "Initial value of num = " <<
num << endl;
```

```
 squareByValue(num);
```

```
 cout << "After squareByValue(), num
= " << num << endl;
```

```
 cout << endl;
```

```
 squareByReference(num);
```

```
 cout << "After squareByReference(),
num = " << num << endl;
```

```
 return 0;}
```